Developing iDSP apps in Python, with the STREAMLIT library

StreamLit is a python library which defines a large set of GUI (graphical user interfaces) to create reactive applications. The most useful are switches, sliders, test inputs. It also redirects the outputs of many python libraries (such as matplotlib) to a web browser.

Added to that, Streamlit-based applications, when installed on GitHub, can very easily be transformed into a publicly available web page.

These two features make StreamLit an ideal candidate for developing iDSP apps. In the next paragraphs, we give a simple example of how to create a simple but effective iDSP app: sinus.py.

1.1 Installing streamlit

Assuming the Anaconda distribution of python is installed on the computer, StreamLit is installed easily [1]by running the Anaconda Prompt app, and typing:

```
>>conda create mystreamlit
```

This will create a new (bare) python environment, which must mow by populated by useful libs:

```
>>pip install streamlit
>>pip install myplotlib
>>pip install scipy
>>pip install soundfile
```

1.2 Creating sinus.py

In a text editor, simply type:

```
import streamlit as st          # This must always be the first line of a streamlit app
from numpy import *
from matplotlib.pyplot import * # Needed for plotting

st.title('My first sine wave')
a=st.slider('Amplitude a', 0.0, 10.0, 5.0)
f=st.slider('Frequency f [Hz]', 0, 1000, 440)
phi=st.slider('Phase phi [rad]', -pi, pi, 0.0)

fe = 10000;                     # These are the three “computational” lines
t = arange(0.0,1,1/fe)
signal = a*sin(2*pi*f*t+phi)

st.latex('''a \sin(2 \pi f t + phi)''')
fig,ax = subplots(figsize=(10,4))
xlim(0,0.010); ylim(-10, 10)
```

[1] A more complex (but more complete) installation procedure can be found at https://docs.streamlit.io/library/get-started/installation.

```
plot(t[0:100], signal[0:100])
xlabel('Time (seconds)')
st.pyplot(fig)
st.audio(signal,sample_rate=fe)
```

There is not much to say about these lines, which are quite self-explanatory.

Notice that no callback function is needed (as is usually the case when creating real-time apps). This is made possible in StreamLit by the fact that, whenever an input GUI is modified by the user, the whole app is run again (keeping all the values from GUIs of course)[2].

This file must then be run inside Anaconda, by typing:

```
>>conda activate mystreamlit
```

Or, for Windows, from the *Anaconda Navigator*, click on *Environments*, then click on the arrow on the right of mystreamlit and choose *OpenTerminal*. Then type:

```
>>cd .\path to the sinus.py file
>>streamlit run sinus.py
```

This will produce the screen shown on Fig. 1, running your app on your local browser, communicating with your local host.

1.3 Deploying the app on Streamlit Community Cloud

If we want the app to be used by others, we must now ask StreamLit to create a web page out of it. We must first create a Github repository for sinus.py.

We have done it here, as an example : https://github.com/thierrydutoit/sine-wave

The repository should include 3 files:

- sinus.py
- requirements.txt, a test file containing the list of libs required by the app:

  numpy
  matplotlib
  soundfile
  streamlit

When this is done, going to the localhost app and opening the *Deploy* menu on the top right will show a "StreamLit Community Cloud" window which will let you define the name of your github repository, and deploy the app automagically.

Another option is to check your https://share.streamlit.io/ page and click on "Create app" on the top right.

Assuming you decide to call it "sine-wave", your app will then be available as:

[2] Since this may lead to excessive and useless computation load, StreamLit has a very efficient way of avoiding parts of the app to be re-run.

https://sine-wave.streamlit.app/

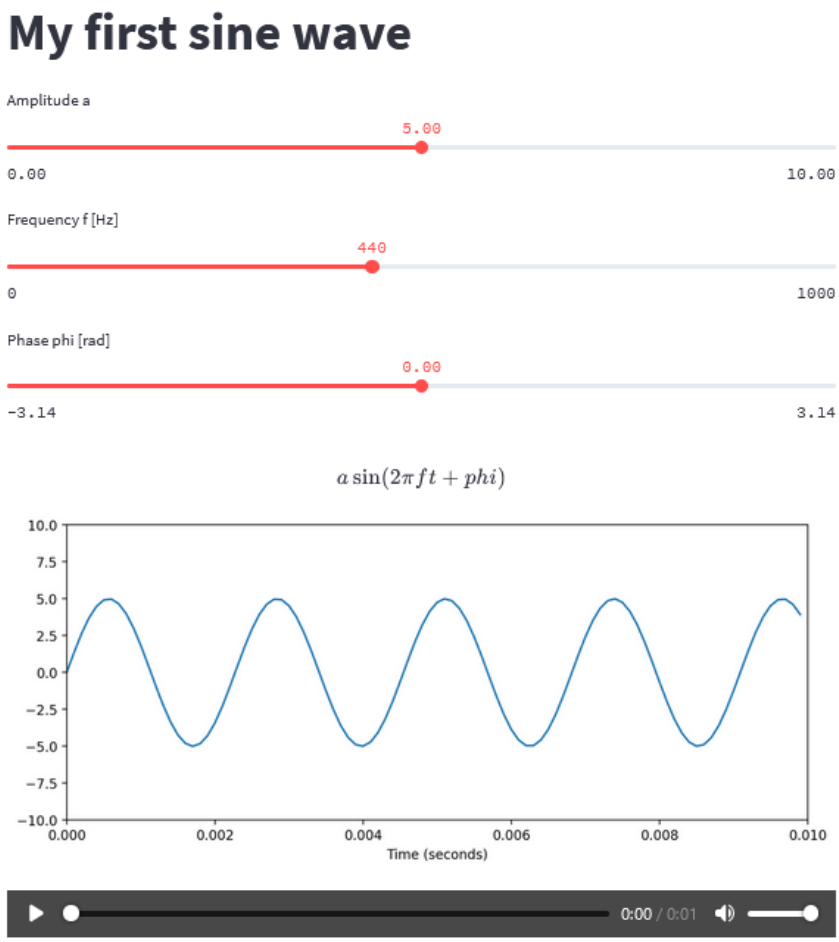

Fig 1. Sinus.py running on the local host web browser

1.4 Deploying the app on Render

As of 2026, Streamlit Community Cloud services seems to be very slow. Frozen streamit apps take hours to wake up, which is clearly unacceptable.

Render is an easy (free - but paid services are also available) alternative in this case. Just create an account on https://render.com/.

Then connect your GitHub account to it, and ask Render to create a Web Service (see https://www.youtube.com/watch?v=4SO3CUWPYf0 for more details if neded).

All your Github repos will appear as candidates for web deployment. Just choose the sine-wave repo, “connect it” by clicking on the Connect button, and set the “Build command” line on the forthcoming page to “streamlit – run sinus.py”.

Finally click on “Create web service”.

Deployment will take 5-15 minutes. When done, your app will be available as https://sine-wave.onrender.com.

Notice that, with the free account, all render apps will go to sleep after 15 minutes of inactivity. Waking them up will again take a few minutes…

1.5 Converting the app into HTML5/JavaScript

Thanks to the fats development of GenAI, it is now relatively easy to create an HTML/Javascript clone of Python/Streamlit apps. Just ask the AI engine to produce a “narrow mode” app, keeping the Streamlit look as best as possible. Also ask it to use Reactive Plots (plots that do not redraw everything when not needed). You may also provide it with some HTML code available from the iDSP web page, as an inspiration. The result is mind-blowing : the app turns into a fully reactive application, in no time.

See https://thierrydutoit.github.io/iDSP/generalized_shannon_theorem.htm for instance.

This is the approach I have followed for many of the apps in the iDSP page.